



Systems Engineering

System Architecture Modeling

Prof. Dr. Kai Höfig

Why Models?



- A model in engineering is a simplified digitalization of the reality.
- To transfer information from the reality into a model costs **money**.

Synchronization problem: *if the reality changes over time, the model should, too. If the model drives the reality, e.g. in engineering processes, the reality has to adopt changes in the model.*

Therefore: *if changes in the reality are costly, stay in the digital model as long as you can until you can be very sure that the system you want to build behaves as required. With the assumption that changes in the model are cheap in general, a system engineering process can be agile.*



- The goal here always is to let the computer help you solve a problem using the model.

One of the primary goals of building intelligent systems is to be able to combine the **computer's capacity for manipulating large amounts of information** with the **human ability to intelligently process information**

Ronald R. Yager

- If there is no problem to be solved with the model or the benefit from automating the problem is less than generating the model, making a model is worthless. *Always challenge modeling activities for this!*



System Architecture



- A **system architecture** represents the **model of a system** including structural, behavioral and collaboration elements together with models derived from it that represent
 1. different viewpoints defined as views on top of the conceptual model,
 2. facets or concerns of the system in dependence on the scope and abstraction level of various stakeholders,
 3. restrictions for the deployment of the system and description of the quality warranties of the system, and
 4. embeddings into other systems.

Hannu Jaakkola and Bernhard Thalheim. (2011) "Architecture-driven modelling methodologies." In: Proceedings of the 2011 conference on Information Modelling and Knowledge Bases XXII. Anneli Heimbürger et al. (eds). IOS Press. p. 98

Goals of Model-based System Architecture



- Check for **feasibility** by the corresponding experts. If no system exists yet, a good architectural model is a better and more efficient way to communicate to experts than written text.
- **Interfaces** that both sides can agree on, the one providing an interface and the one consuming an interface.
- **Validating** the system from different points of view, e.g. structural, behavioral and collaborative
- **Decrease the complexity** of a system by divide-and-conquer principle and abstractions.



Modeling an architecture costs money. Is it worth it? Are there benefits from modeling an architecture or is it sufficient to just write down a documentation or requirements?

Syntax and Semantics of a Modeling Language



- **Syntax** is the set of **rules**, principles, and processes that govern the structure of a language.
- **Semantics** is the meaning of a language and the **relationship** between signifiers like words, phrases or signs of their language and what they stand for in reality.
- Without semantics, a graphical modeling language is just drawing.
- Without syntax, a modeling language is.. modern art?

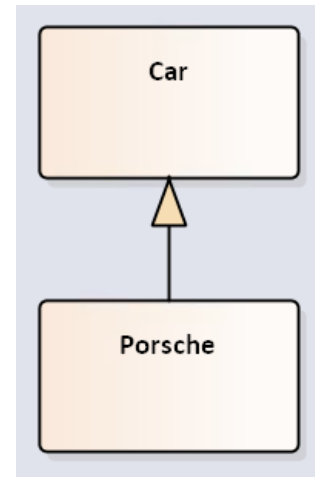
M2

Semantics

M1

Semantics

M0



Syntax
UML Class
Diagram

```
public class Car {  
}  
public class Porsche extends Car {  
}
```

Syntax
Java 8



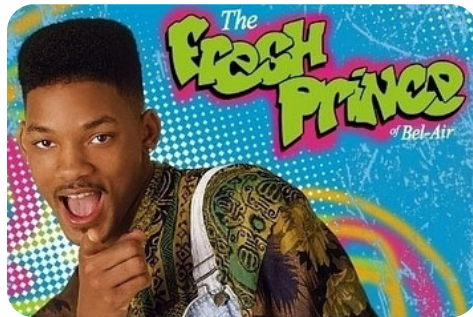
Syntax
Law of thermodynamics

UML2

- Current Version 2.5.1 (Dec 2017)
- Graphical modeling language to develop software, software intensive systems and other systems.

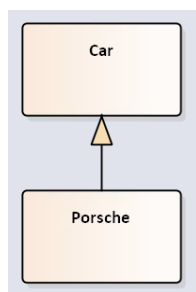


- From the 90s



- This language has a strong syntax. → But no defined semantics ←
Missing semantics come from the fact, that the language is *universal*. It was not defined to explicitly generate java code from it, but do allow modeling for any language or software intensive system.

So this:



```
public class Car {
}
public class Porsche extends Car{
}
```

is more common sense than
being standardized

SysML

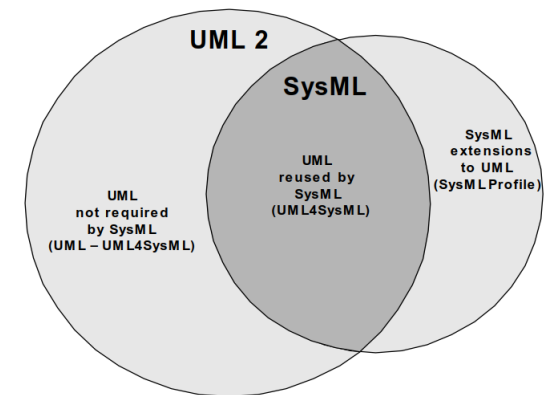


- Current Version 1.6, 2019 (1.7beta 2022, 2.0 in preparation)
- Graphical modeling language for Systems Engineering to tackle complexity

- From 2007, 1st iPhone



- Again: Syntax but no semantics
- SysML uses a part of the already existing elements of UML, but not all of them.
- SysML extends the UML by defining new elements that are not part of the UML using MOF and the UML profiling mechanism.



Diagrams in SysML



- **Requirement Diagram** (req), Requirements and decomposition
- Structural Diagrams
 - **Block definition diagram** (bdd), static structure and decomposition of a system
 - **Internal block diagram** (ibd), internal view of a block
 - **Package diagram** (pkg), structure of modeling elements
 - **Parametric diagram** (par), parameter constraints in the system
- Behavioral Diagrams
 - **Activity diagram**
 - **Use case diagram**
 - **State machine diagram**
 - **Sequence diagram**



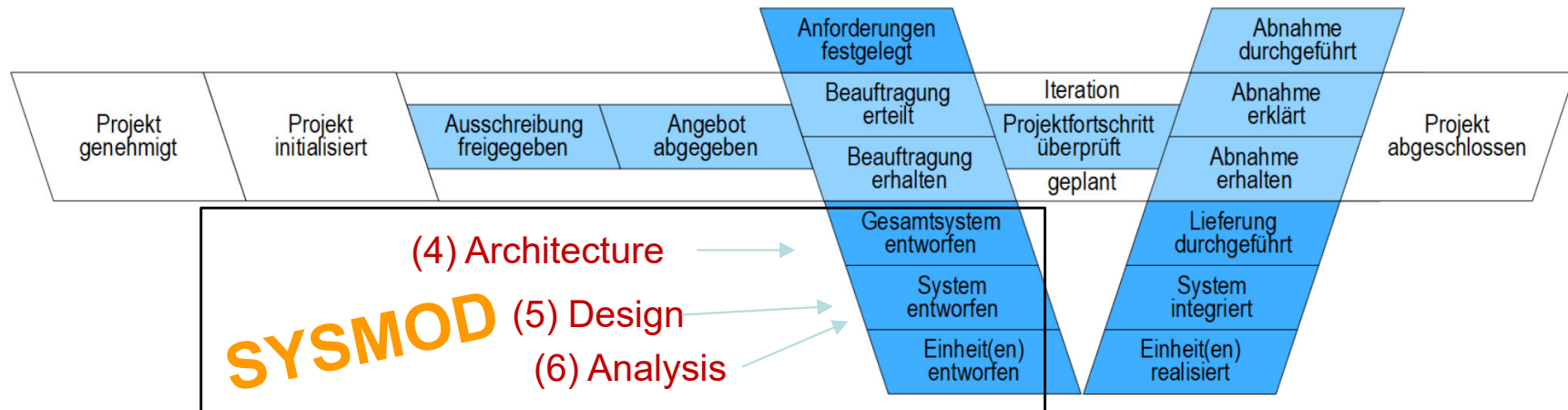
SYSMOD – A purpose-driven modelling process for Systems Engineering with SysML

- ◆ Central micro-process:
 - 1) Item identification
 - 2) Blackbox item definition/specification (context)
 - 3) Deep dive, White box/Gray box item specification

- ◆ Roles
 - 1) Systems Engineer has two central roles:
 - 1) Requirements Engineer, responsible for requirements and analysis model
 - 2) System Architect, derives architectures from requirements
 - 2) System Tester
 - 3) Project Manager



What's next?



- ◆ The *Business or Mission Analysis*, *Stakeholder Analysis* and *Requirements analysis* of the **INCOSEs Technical Systems Engineering Process** *are done*.
- ◆ Now we take a shot at *Architecture* and *Design* using **SysML**



SYSMOD Architecture Process

Different Architectures known to SYSMOD. An architecture is the sum of elements describing a system, that are fundamental. Changing them is hard in later phases of the system engineering process.

1. **Base Architecture** (*part of „SYSMOD analysis process“*)
Every project has its constraints (technological, cultural, legislative) that result in a certain way things have to be. These essential predefined elements form a base architecture.
2. **Functional Architecture** (*part of „SYSMOD analysis process“*)
Functions form the primary goal of a system. They can be reused over different iterations and are independent from technical solutions. Can be found in use cases and is specified in activities
3. **Logical Architecture**
Here, specific **technical concepts** and principles are described, that are intended to be used to provide a successful technical concept for a system's goals.
4. **Physical Architecture**
Further refining the logical architecture, specific **physical blocks** are here used to **implement** what's defined in logical blocks. Elements from both architectures can be mixed in one diagram.



SYSMOD Analysis Process

1. Define system idea and goals
Description of basic principles, purpose of the system
2. Define base architecture
Define technological constraints and implications for the project.
3. Requirements engineering includes stakeholder analysis
The stakeholder analysis provides the sources of requirements.
4. Modelling system context (interfaces of the system)
The system interacts at its **boundary** with external **actors** (people or systems) and exchanges **objects** via its **interface**
5. Define use cases that further describe functional requirements
Use cases are free of redundancies and precisely described using activity diagrams
6. Identify domain blocks
Domain blocks are things that the system is aware of. This step aims at summarizing those domain blocks along with their properties.



Base and Functional Architecture is the Requirement Specification

1. **Base Architecture** (part of „SYSMOD analysis process“)
Every project has its constraints (technological, cultural, legislative) that result in a certain way things have to be. These essential predefined elements form a base architecture.
2. **Functional Architecture** (part of „SYSMOD analysis process“)
Functions form the primary goal of a system. They can be reused over different iterations and are independent from technical solutions. Can be found in use cases and is specified in activities

The **requirement specification (Lastenheft)** is the architectural element that specifies the requirements of the system. Formerly this was a textual document that was handed over to the contractor to implement the system.

Here, this is a model containing the base and functional architecture as a SysML model developed by contractor and client, e.g. using an agile approach.

It is done, when a contractor fully understands **what** to do.



Logical and Physical Architecture is the Target Specification

3. Logical Architecture

Here, specific **technical concepts** and principles are described, that are intended to be used to provide a successful technical concept for a system's goals.

4. Physical Architecture

Further refining the logical architecture, specific **physical blocks** are here used to **implement** what's defined in logical blocks. Elements from both architectures can be mixed in one diagram.

The **target specification (Pflichtenheft)** is the architectural document that describes the implementation of the requirement specification. It was text, now is also a model. It is something that contractor and client agree upon for a legal contract.

Here, this is a model containing the logical and physical architecture as a SysML model developed by contractor and client, e.g. using an agile approach.

It is done, when a contractor fully understands **how** to do it.



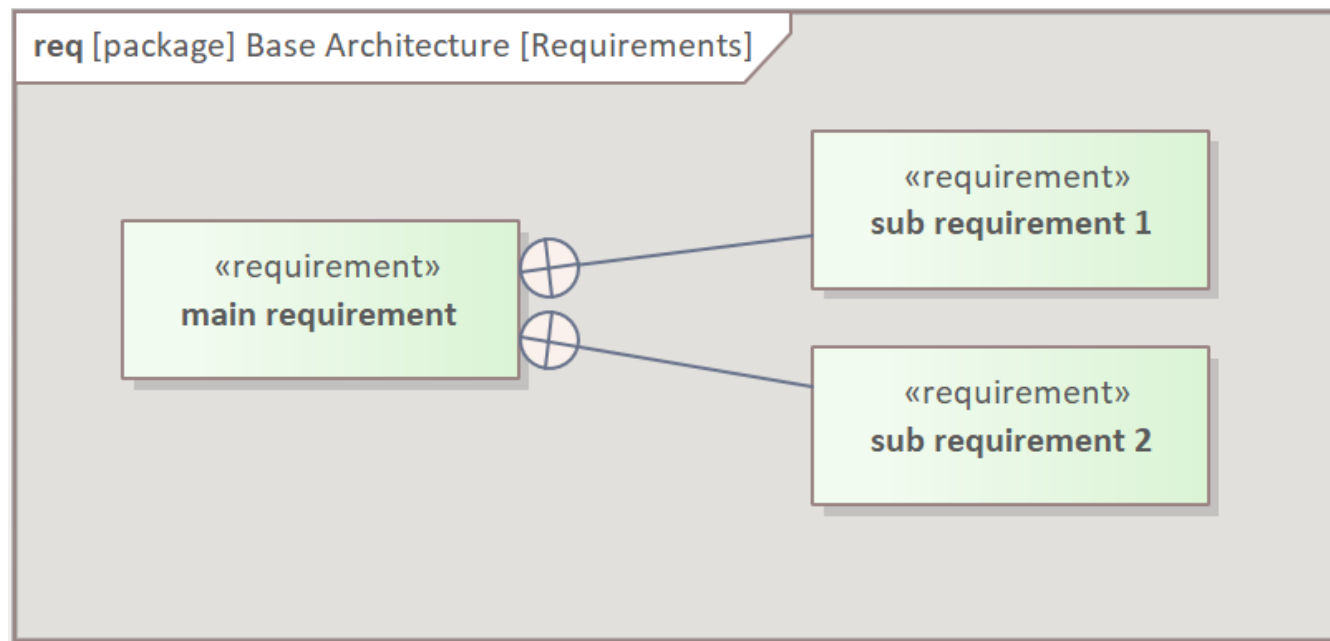
THROs MiniSYSMOD Process

- ◆ The SYSMOD Process is a process that also fits large and complex systems. Furthermore, SysML 1.7 and even more SysML 2.0 contain a large amount of modelling elements to deal with this complexity.
- ◆ The THRO MiniSYSMOD process is a narrowed down version to teach the basic principles to model systems. It can be used for small to mid sized engineering projects including software engineering or software intense systems.
- ◆ The analysis process and architectures stay the same as in SYSMOD. We also use SysML, but only a subset fitting many purposes that can extended by yourself once you advance in modelling.



Phase 1: Base Architecture

- ◆ Contains central requirements in a magnitude that the enable to grasp the main purpose of the system such as
 - opscon and conops in the central main requirement as system purpose
 - sub requirements to model constraints
- ◆ We use the SysML requirement diagram (**req**) to do so.

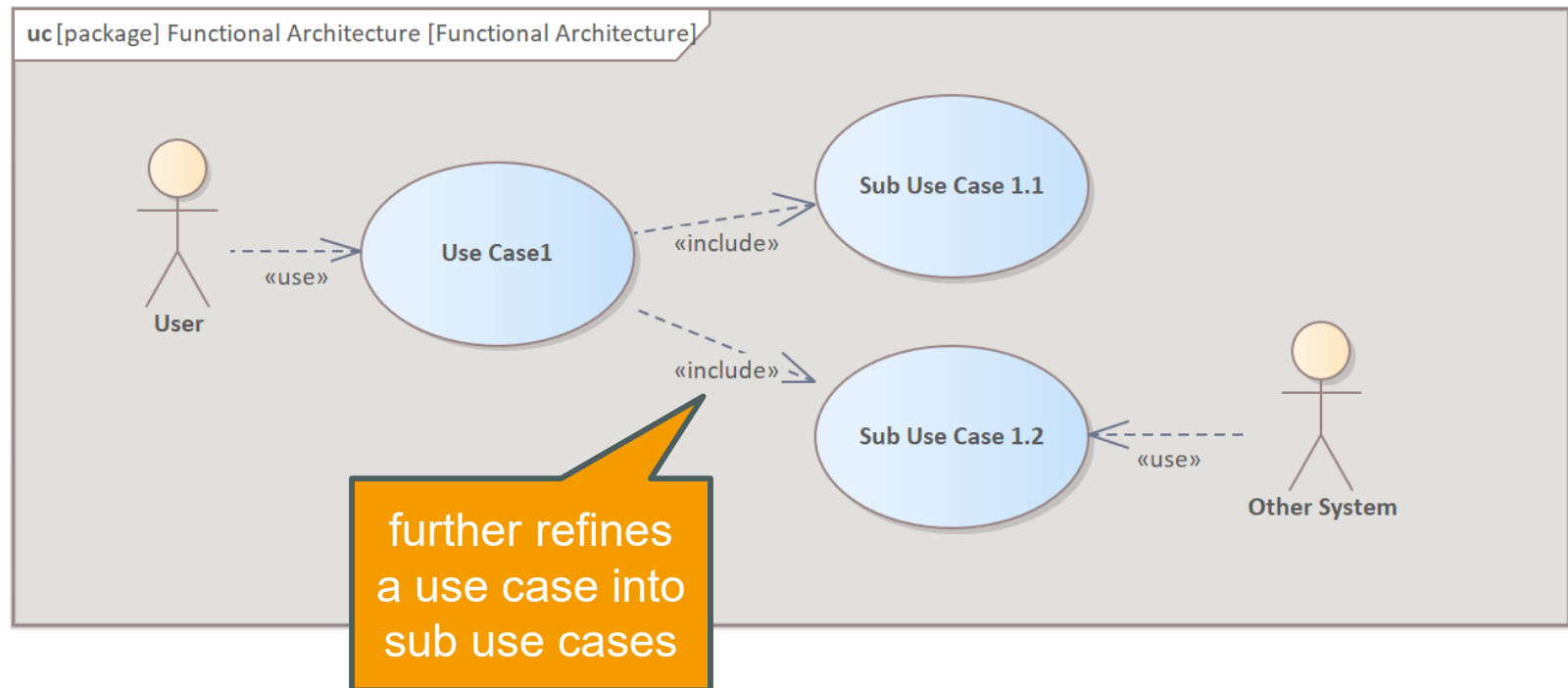


arrow:
main requ consists of
sub 1 and 2



Phase 2: Functional Architecture – Usecases

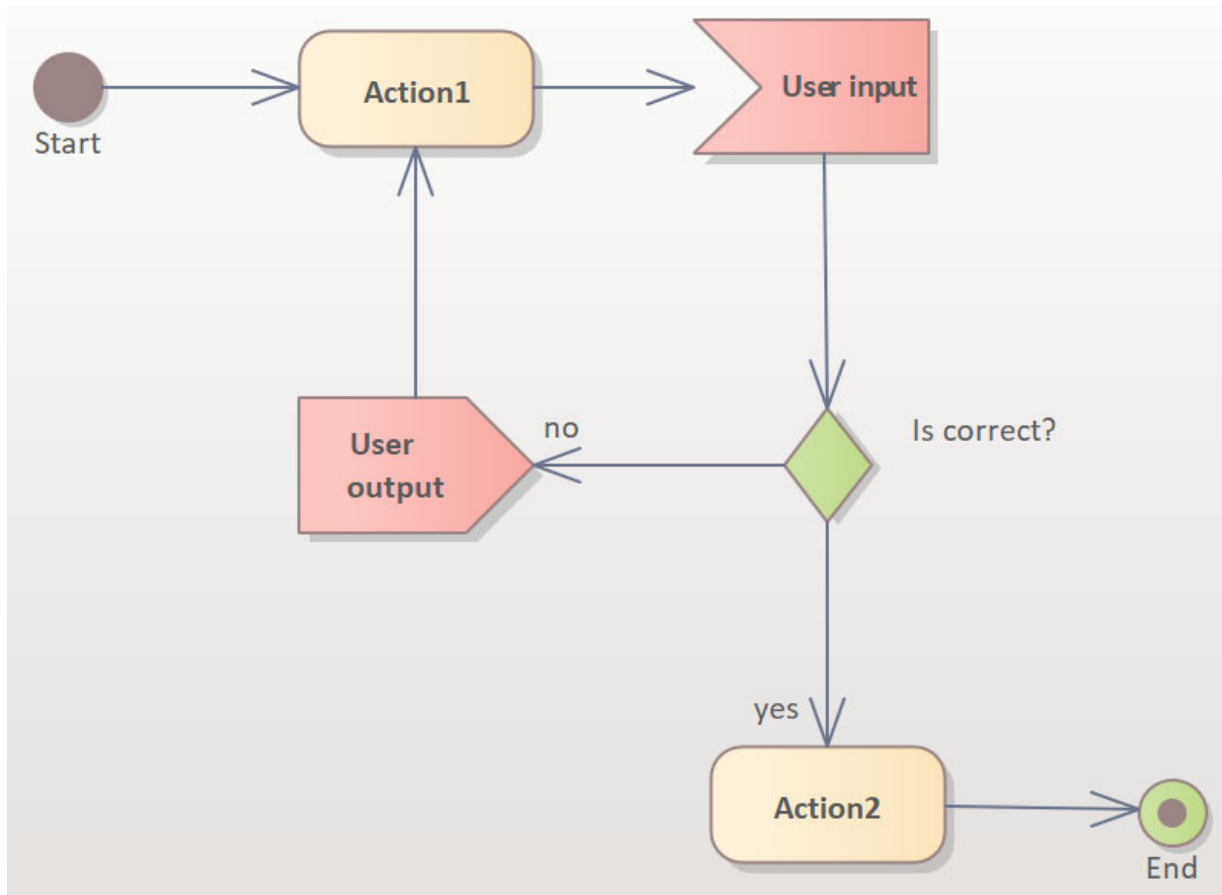
- ◆ Contains the functions (independent of implementation) of the system such as the actors, that are either users or other systems interacting with the system in a use case Diagram (uc).





Phase 2: Functional Architecture – Activity Diagram

- ◆ Activities describe a functional requirement provided by a use case in detail in an activity diagram (**act**).

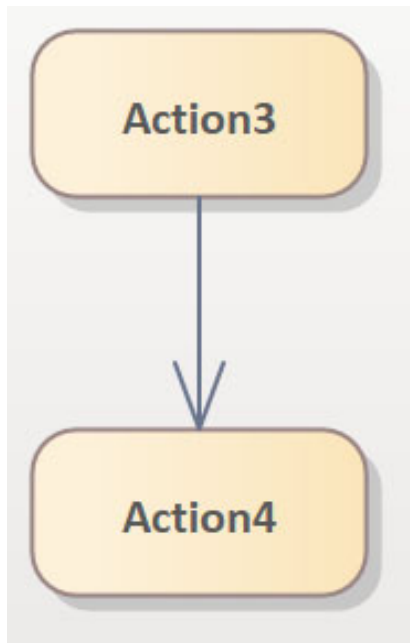


We model only what is important to the contractor / implementation!

each use case gets its own activity diagram



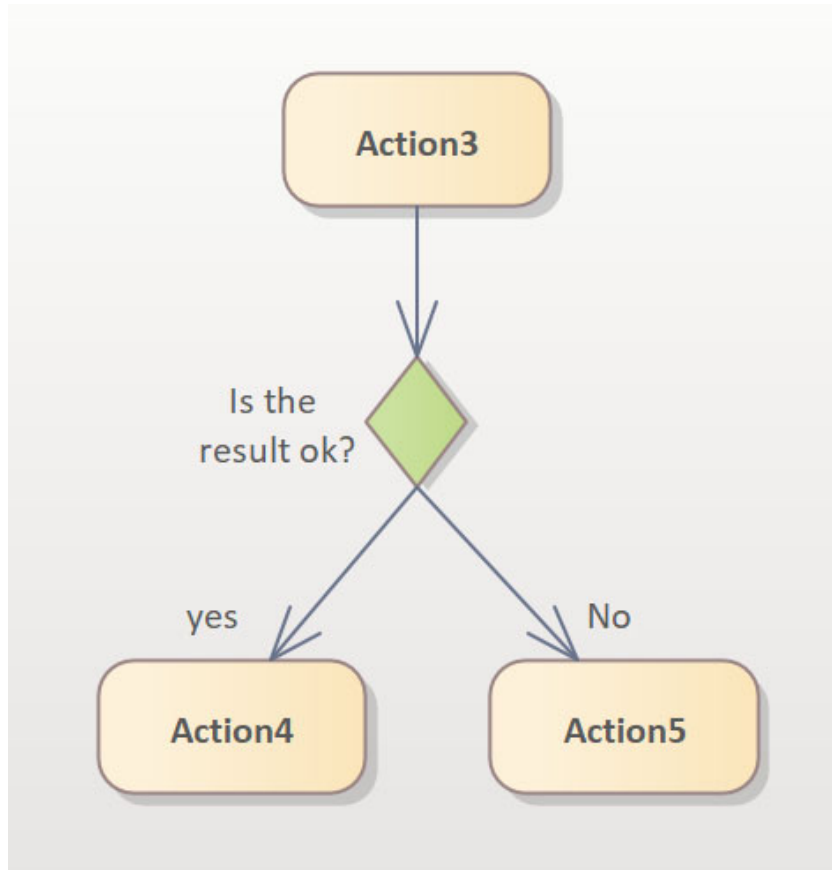
Activity Diagram – Actions



- ♦ An **atomic action** is something that describes a part of an activity that is fully understood and that does not require further description.
- ♦ A **control flow** connects two actions and is taken when an action is done.
- ♦ Parallelism is possible.



Activity Diagram – Decisions

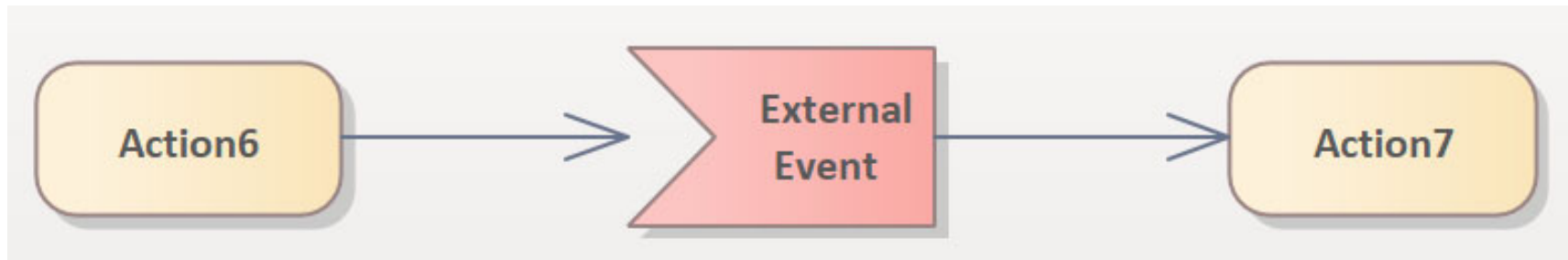


- ♦ A **decision** is evaluated after the action has finished.
- ♦ The outcomes of a decision should be mutual exclusive and complete.

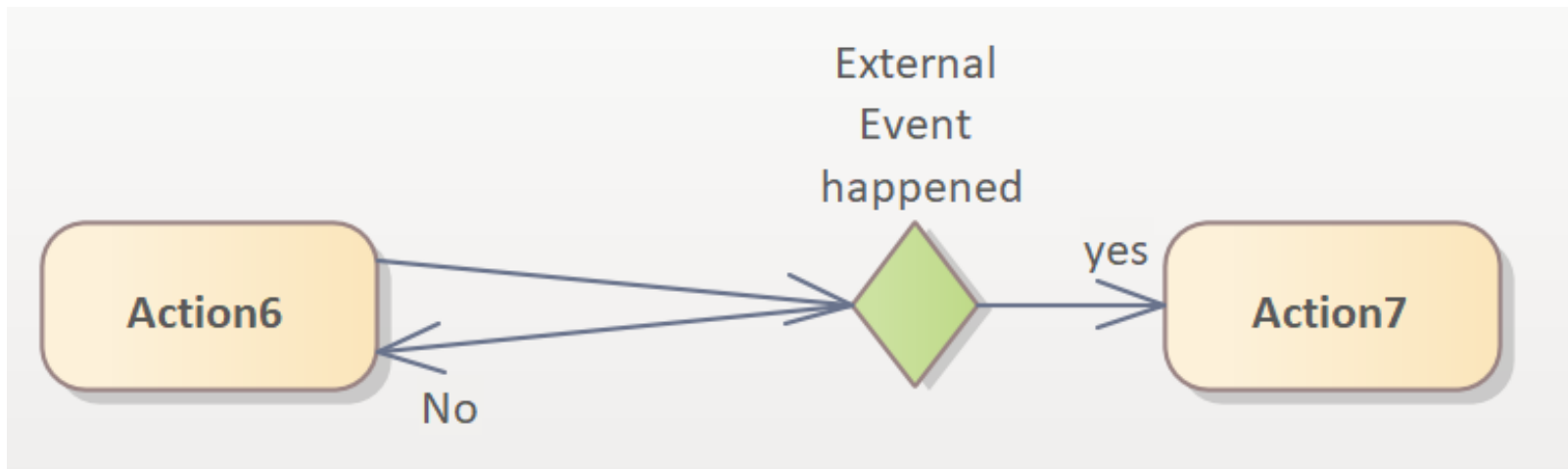


Activity Diagram – Receiving events

- ◆ A **Receive** Event triggers the outgoing control flow and is only recognized when its incoming control flow is connected to the active action. It can be used to model user inputs.



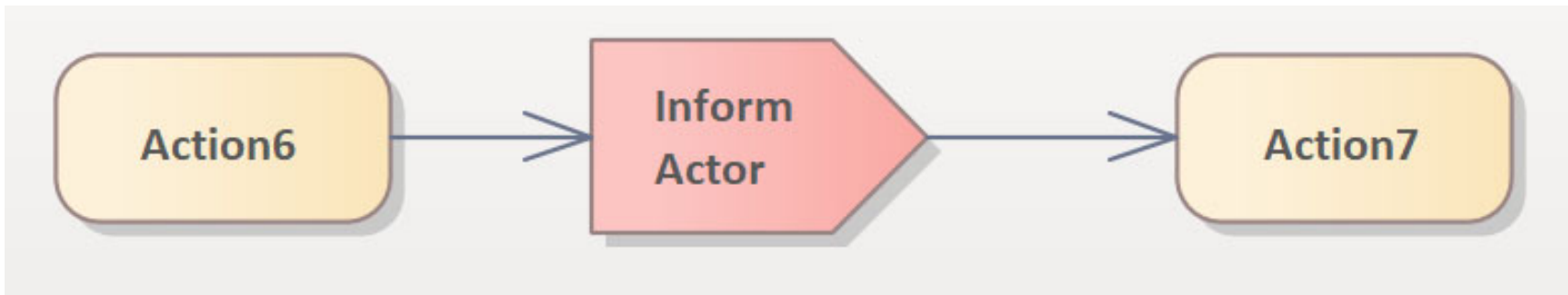
- ◆ When **Action 6** is executed and the **External Event** is happening, **Action 7** is executed



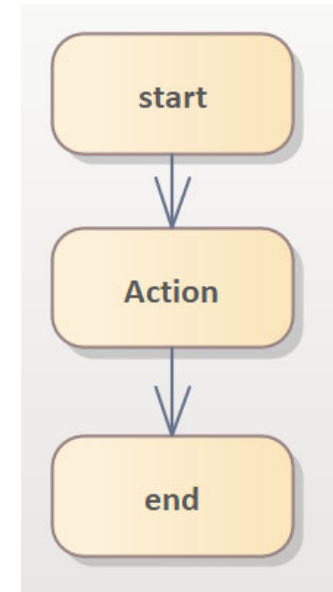
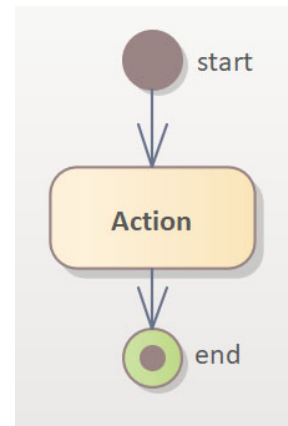
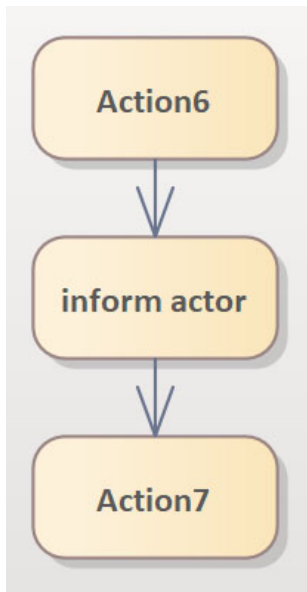


Activity Diagram – Sending events

- ◆ A **Send Event** is something that is being send out.



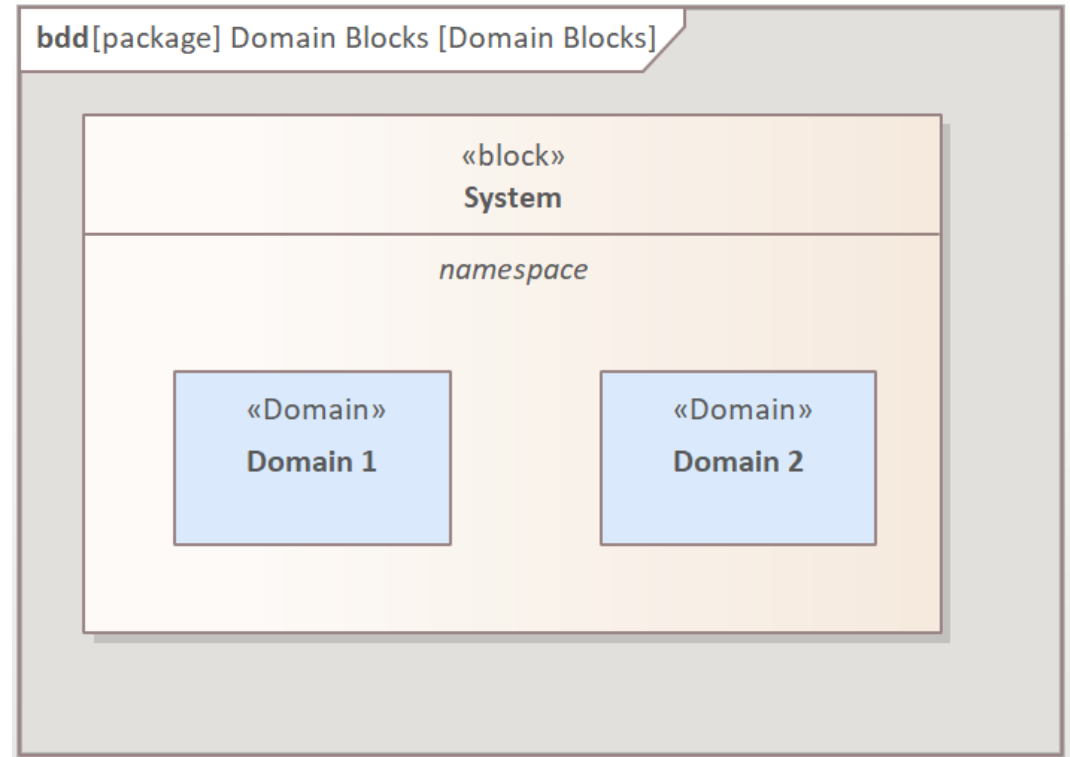
- ◆ It could also be an action. Same is true for **initial** and **final** actions.





Domain Blocks

- ◆ Domain blocks identify the main domains that a system addresses. Again, as a part of the system architecture, we completed identifying domain blocks, when we can be sure, that we have all domains that are required to build the system (we are sure that we can build the system with them, not how). *Do we have the knowledge to build the system?*
- ◆ For domain blocks, we use a block definition diagram (bdd). The root element is the system and parts of the system are domain blocks with the stereotype <<Domain>>
- ◆ The diagram is placed in the base architecture.





Enterprise Architect

- ◆ Create new empty model with no wizard
- ◆ Select SysML as perspective
- ◆ Add View → package only on the model to create base architecture and functional architecture for your model.
- ◆ For e.g. your base architecture add diagram and select SysML 1.5 → Requirement Diagram.

